



PREVENTING AND FIGHTING EARLY LEAVING
FROM EDUCATION & TRAINING WITH AI

stAI

Act before it happens: Preventing and fighting Early
Leaving from Education and Training with Artificial
Intelligence

T4.4 ELS-PREDIN System User Manual

Author:



ARTIFICIAL
INTELLIGENCE
RESEARCH GROUP



Co-funded by
the European Union

PROJECT INFORMATION

Project acronym:

stAI

Project title: Act before it happens: Preventing and fighting Early Leaving from Education and Training with Artificial Intelligence

Project N°:2023-1-IS01-KA220-SCH-000157308

Sub-programme or KA:

KA220-SCH - Cooperation partnerships in school Education

Website: www.stai-edu.eu

Consortium:



SCHOOL OF EDUCATION



ARTIFICIAL
INTELLIGENCE
RESEARCH GROUP



Colegiul Național de Artă
Octav Băncilă
Iași



Table of Contents

1. Introduction	4
2. System overview	4
2.1 Data pipeline	5



Co-funded by
the European Union

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Education and Culture Executive Agency (EACEA). Neither the European Union nor EACEA can be held responsible for them. **Project N°:2023-1-IS01-KA220-SCH-000157308**

2.2 Model layer	5
2.3 Explainability layer	6
2.4 Visualization and output layer	6
3. Main functionalities	6
3.1 Data ingestion	6
3.2 Data exploration	6
3.3 Model training	6
3.4 Prediction	7
3.5 Explainability	7
3.6 Output and reporting	7
3.7 Responsible-use support	7
4. Non-functional requirements	7
4.1 Privacy and data protection	7
4.2 Local explanation generation	7
4.3 Deployment and isolation	7
4.4 Usability and responsible use	8
4.5 Scalability across countries	8
5. Getting started	8
6. User interface guide and workflow	8
6.1 Uploading data	10
6.2 Exploring datasets	12
6.3 Training models	17
Logistic Regression	17
Decision Tree	17
Random Forest	18
Neural Network	18
XGBoost	18
6.4 Generating predictions	21
Manual entry	23
Upload a CSV file	28
Interpreting predictions	29

1. Introduction

ESL-PREDIN is a data-driven prototype designed to help educational stakeholders identify students at risk of early school leaving and to support evidence-based prevention across student, school, and regional levels. The system is intended for school staff, guidance teams, prevention specialists, and educational authorities who work with student data and support early-intervention strategies.

ESL-PREDIN combines predictive modelling with automated insight extraction, enabling users to interpret historical educational data and make informed decisions. Its development focuses on two core objectives: (1) creating accurate and explainable models that predict students at risk and (2) generating actionable insights from past data to guide preventive measures.

Building on these objectives, the development of ESL-PREDIN focused on creating a practical, web-based prototype that schools or education departments could use to identify students at risk of early school leaving and to explore the underlying patterns in their data. The system brings all core functionalities together in one place: users can upload a dataset, examine its structure, train a model, and later apply that model to generate predictions. Because datasets from different countries vary considerably, the workflow was designed to be flexible, allowing each dataset to pass through the same process while generating a dedicated model tailored to its own features. This was essential, as the project worked with data from Belgium, Iceland, Italy, Portugal, and Romania—each reflecting different educational systems and social contexts. Beyond prediction, the system enables users to explore trends and risk groups within their data, acknowledging that early school leaving is rarely driven by a single factor. Finally, every prediction is accompanied by an explanation and a short, actionable summary, ensuring that the tool supports human decision-making rather than replacing it, and helping teachers interpret results and consider appropriate next steps.

2. System overview

ESL-PREDIN is built around a modular architecture that processes educational data, trains predictive models, generates explanations, and presents results in a clear and actionable format. The system is organised into four main layers: the data pipeline, the model layer, the explainability layer, and the visualization and output layer, as shown in Figure 1. Together, these components support the system's core goals: enabling flexible data ingestion across countries, producing accurate and explainable predictions, and presenting insights that help educational staff make informed decisions.

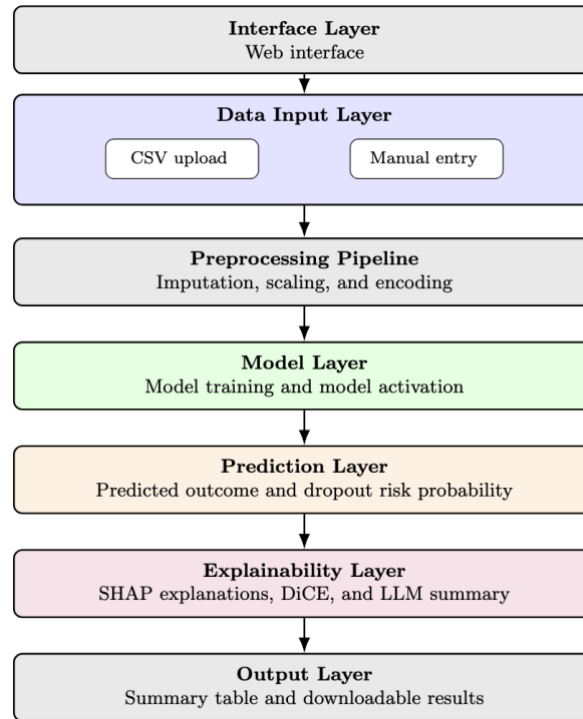


Figure 1. Overview of the proposed system architecture. The point to notice is that the design is layered rather than monolithic: data flows top to bottom through separate interface, preprocessing, model, prediction, explainability, and output layers, so the same architecture can be reused across countries while each dataset keeps its own feature structure.

2.1 Data pipeline

The data pipeline manages how datasets enter and move through the system. Users begin by uploading a structured educational dataset. Because each country’s dataset contains different variables, the system preserves the original feature structure rather than forcing a unified schema. During model training, the user selects a target variable, which is separated from the input features. The features then pass through a preprocessing workflow: numerical variables are imputed and scaled, while categorical variables are imputed and encoded. This ensures that mixed educational data—containing both numbers and categories—can be processed consistently. When making predictions, the system uses the stored feature list to align new student records with the active model, guaranteeing compatibility between the dataset and the model’s structure.

2.2 Model layer

The model layer handles the training, storage, activation, and application of predictive models. The system supports multiple machine-learning classifiers, including logistic regression, decision trees, random forests, XGBoost, and neural networks. After training, each model is saved together with its metadata: country, dataset name, target column, selected features, model type, and evaluation metrics. Only one model is active at a time, ensuring that predictions are made using the correct feature structure. Because datasets differ across countries, the system follows a country-specific modelling approach rather than attempting to build a single universal model.

2.3 Explainability layer

The explainability layer provides insight into why a prediction was made. It generates feature-based explanations that highlight the variables most responsible for the predicted outcome. It also produces counterfactual explanations, which suggest realistic changes that could alter the prediction—focusing on factors that schools can meaningfully act upon. To make these insights accessible, the system converts structured explanation outputs into natural-language summaries using a local language model. All explanation generation happens on the user’s machine, aligning with the project’s privacy requirements and ensuring that student-level data never leaves the institution.

2.4 Visualization and output layer

The final layer presents results to the user. For data exploration, the system provides visual summaries that help identify patterns, distributions, and dropout-related trends. For predictions, it displays a summary table containing the student identifier, predicted outcome, dropout-risk probability, explanation, and suggested action. Outputs are available in two formats:

- Summary output, designed for everyday use by teachers and school staff.
- Technical output, containing full prediction details, feature contributions, and counterfactual explanations for deeper analysis or review.

This separation ensures that different user groups—teachers, guidance teams, researchers, or AI experts—can access the level of detail they need.

3. Main functionalities

The functional requirements outline the core capabilities that the ESL-PREDIN prototype must provide. They were defined based on the objectives of the stAI project, the structure of the multi-country datasets, and the feedback collected during the Brussels training and stakeholder discussions. During that session, participants shared how they expect the tool to be used, how predictions should be interpreted responsibly, what types of inputs and outputs are most useful, and why clear explanations are essential for supporting decision-making in schools. Based on these insights, the main system functionalities are as follows:

3.1 Data ingestion

The system allows users to upload structured educational datasets and use them for data exploration and model training, enabling predictions. Because datasets come from different countries, the system must preserve the original feature structure of each uploaded dataset.

3.2 Data exploration

The system enables users to inspect the dataset through visual summaries. This helps identify patterns, distributions, and potential risk-related groups either before or after model training.

3.3 Model training

The system allows users to select a dataset, country, target column, and machine learning model. Once trained, the model and its metadata must be stored so they can be activated later for prediction.

3.4 Prediction

The system generates predictions for individual students or for multiple records at once. Users are able to upload a file containing several student entries and receive predicted outcomes with associated dropout-risk probabilities.

3.5 Explainability

The system provides explanations alongside each prediction rather than returning only a class label. Explanations highlight the main factors influencing the prediction and offer intervention-oriented suggestions to support decision-making.

3.6 Output and reporting

The system presents prediction results in a clear summary table and allow users to download the outputs. A practical summary is provided for everyday interpretation, while a technical output preserves detailed model and explanation information.

3.7 Responsible-use support

The system clearly communicates that predictions are intended to support—not replace—human judgement. Outputs should help educational staff reflect on possible risks and interventions, but final decisions must always remain with human professionals.

4. Non-functional requirements

In addition to defining what the system must do, requirements for how the system should behave were also identified. These requirements focus on qualities such as privacy, deployment, usability, and responsible use. They strongly influenced the design choices behind the ESL-PREDIN prototype and ensure that the tool can be used safely and appropriately in real educational environments.

4.1 Privacy and data protection

Privacy was a primary concern. Because the prototype processes student-level information—such as academic progress, attendance, family background, or socio-economic indicators—the system was designed around a simple principle: all data must remain within the institution that owns it. This reflects stakeholder feedback, where privacy risks were highlighted as a major challenge when linking educational data sources. As a result, model training is intended to occur locally, with each school, region, or country working only with its own data.

4.2 Local explanation generation

The same privacy considerations guided the approach to explanation generation. All explanatory text is produced locally using an on-device language model, ensuring that prediction results and student details are never sent to external AI services. This keeps sensitive information off third-party servers and aligns with strict data-governance expectations in schools and educational systems.

4.3 Deployment and isolation

A single centralised system for all countries and schools would be difficult to manage and incompatible with varying data structures and privacy rules. A separate deployment per school or institution is more appropriate. Containerised deployment (e.g., via Docker) was identified as a suitable direction. This setup

ensures that each school works only with its own data, trains its own models, and generates predictions locally, preventing data leakage across institutions or countries.

4.4 Usability and responsible use

The system must be understandable and usable for non-technical staff. Predictions should be presented in a clear, reflective format rather than as raw technical outputs. For this reason, the prototype provides readable summary tables, plain-language explanations, and optional technical details for advanced users. Importantly, the system is designed to support—not replace—human judgement. Stakeholders emphasised that automated labelling can be harmful, and that the tool should assist teachers and feed into existing support processes rather than act as a decision-maker.

4.5 Scalability across countries

Scalability in this context refers to the ability to work with diverse educational systems rather than handling large volumes of data. Because the datasets differ in structure and level of detail, the system uses a shared workflow while preserving each country's feature set. This makes the prototype flexible and adaptable, avoiding the limitations of tools tied to a single fixed data format.

5. Getting started

To begin using the system, access the project repository at the following GitHub link: <https://github.com/leticiaarco/ELS-PREDIN-APP>

All installation steps, access instructions, and initial setup procedures are detailed in the repository's README files. Users should follow these instructions carefully to ensure the system is installed and configured correctly before running any components.

6. User interface guide and workflow

The interface was kept plain on purpose. The home page shows the system as a few cards: one for early risk prediction, one for data exploration, and a separate panel for model training. Figure 2 shows the home page as it appears on start-up, whereas Figure 3 shows the model-training section in its expanded form to illustrate the underlying steps that enable exploration and prediction.

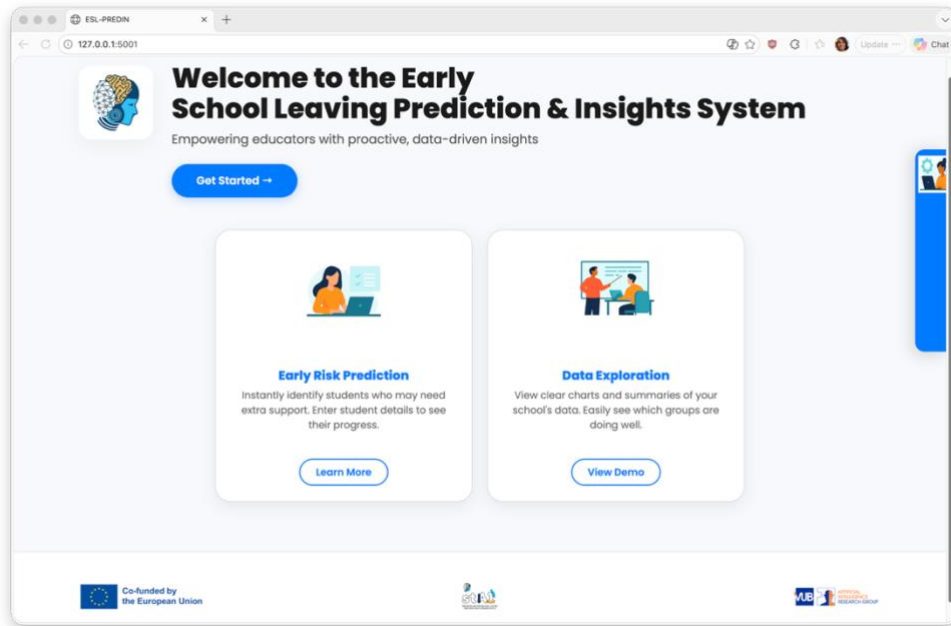


Figure 2. The home page presents the workflow through three simple entry points—early-risk prediction, data exploration, and model training. The first two are the main user-facing modules, while model training appears more discreetly, as it is a supporting function that enables both exploration and prediction.

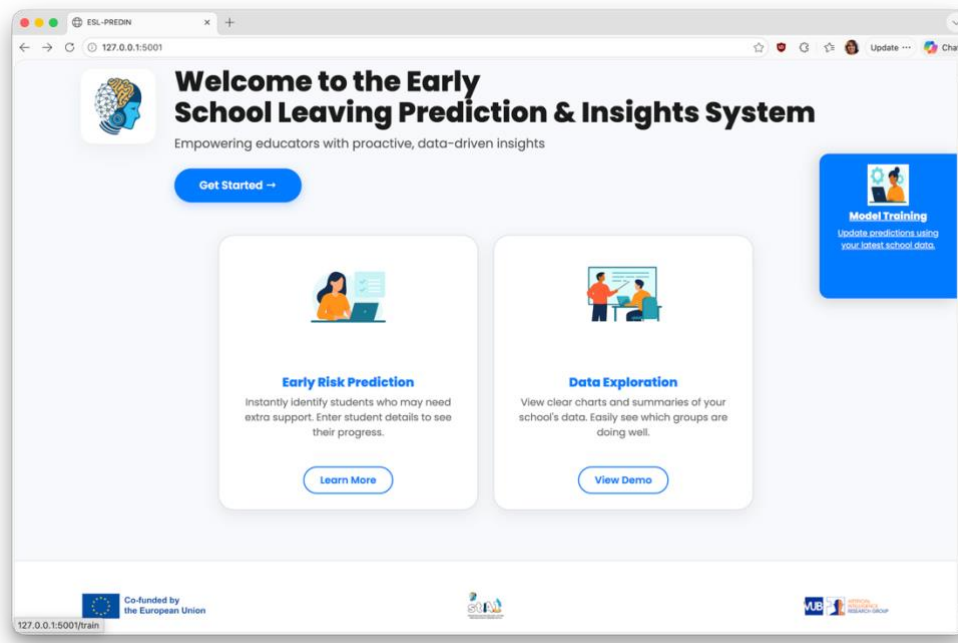


Figure 3. The model-training section is shown in its expanded form to illustrate the underlying steps that enable exploration and prediction.

The pages are ordered the way the system is actually used. The user uploads a dataset, describes its main columns, explores it, trains a model, activates it, and then uses it to predict. Each step feeds the next. Figure 4 lays this out as two rows: the top row is what the user does, and the row below is how the system

answers each action. Reading it left to right gives the whole journey, from uploading a file to downloading the results. This is the typical order for a first run rather than a fixed sequence: exploration is optional, a user who already has a trained model for the country can go straight to prediction, and training can be repeated to compare or switch models. Figure 4 shows the complete path so that every step and its system response is visible in one place.

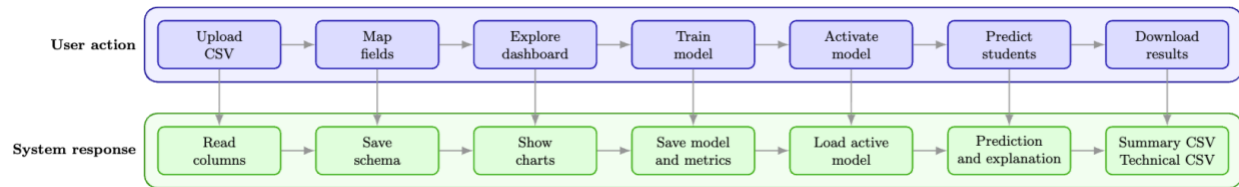


Figure 4. The same workflow read as two parallel lanes: every simple user action in the top lane (upload, map fields, explore, train, activate, predict, download) triggers one system response directly below it (read columns, save schema, show charts, store model, load active model, explain, export). The user only ever performs lightweight steps while the system carries the technical work, and that the active-model step is the single hinge on which every later prediction depends.

Figure 4 makes the split between the two roles clear. The user only takes simple actions: pick a file, map a few fields, look at the dashboard, train and activate a model, ask for a prediction, and download what comes back. Each of those is paired with the work the system does underneath. When the user uploads a CSV the system reads its columns; when the user maps the fields the system saves that schema; training stores the model with its metrics, and activation loads it as the one model in use. The two heavier steps sit on the right: a prediction comes back together with its explanation, and the download step produces both the summary CSV and the technical CSV. The rest of this section walks through these steps in the same order.

6.1 Uploading data

Users can upload a dataset in two ways: through the Data Exploration module or through the Model Training section. When accessing the Data Exploration module, the interface shown in Figure 5 appears. Here, the user selects the CSV file containing the historical annotated data used for discovering insights or for training classifiers.

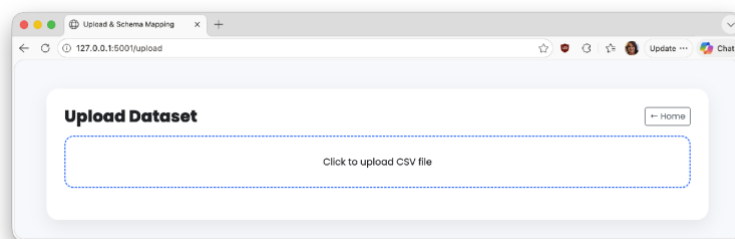


Figure 5. Here the user should select the CSV file containing the historical annotated data used for discovering insights or for training classifiers.

Figure 6 shows an example of such a dataset, where each row represents a student and each column corresponds to a feature (a variable or characteristic) describing all students in the dataset. It is important to note that previous classification of students is required. The system learns from these annotated

examples to make predictions for new incoming students. Without labelled historical data, the model cannot be trained or used for risk prediction.

</

Figure 6. Excerpt of a dataset where each row corresponds to a single student and each column represents a feature (variable or characteristic) describing that student. Note that the previous classification of students (enrolled or dropped out) is required for model training.

Upload Dataset

Click to upload CSV file

Uploading: ESL_Belgium.csv

Select the column that shows each student's final status (Dropped Out or Enrolled)

target

Dropout Value: dropout

Enroll Value: enroll

Continue to Explore →

Figure 7. Mapping the target column and its values. This step tells the system which column contains the dropout label and how its values correspond to “dropout” or “enrolled,” allowing it to work with datasets that use different names and labels.

The country datasets are not identical, so the system does not assume a fixed set of features. Instead, it asks the user to map a few key roles: which column is the target, and which target value means dropout,

and which means still enrolled, as shown in Figure 7. That mapping is what lets the same system work across the different country datasets, even though they use different column names and labels. Once the columns are known, the user can open the exploration page. The system shows summaries and charts of the uploaded data, so the user can look at distributions and dropout patterns before training anything, as described in the next subsection.

6.2 Exploring datasets

The dashboard is the part that supports data exploration. It is there to help the user look at the dataset before or after training. The dashboard reads the active dataset together with the column mapping from upload and shows it through summary figures and interactive charts. The charts were built with Plotly, which gave simple interactive visuals without much setup. The page is long, so it is shown here in three parts that follow the order a user scroll through.

At the top of the page, the system provides a quick overview of the dataset, as shown in Figure 8. It displays the total number of students, along with the counts and proportions of those who dropped out and those who remained enrolled. In the Belgian dataset illustrated here, the system reports 71,667 students in total, of whom 9,449 dropped out (13.2%) and 62,218 stayed enrolled (86.8%). The three cards are colour-coded, with dropout in red and enrolment in green, so the balance between them is clear before reading any chart.

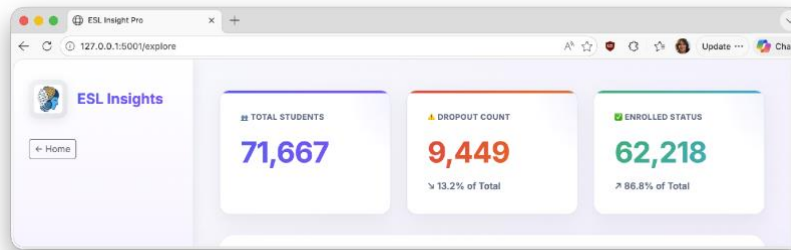


Figure 8. Summary of the dataset showing total students, dropout count, and enrolled count with their proportions.

Right below that is the **Top Risk Segments** chart, which the system fills automatically: it scans the features and picks out the student groups with the highest dropout rates, drawn as horizontal bars and coloured by severity (red for very high, down through orange and yellow to green for low). Each bar also shows the rate and the count behind it, as shown in Figure 9. That count is worth keeping an eye on, because some of the highest-rate groups are also small, and a near-total dropout rate over a handful of students does not mean the same thing as a high rate over a large one. Under the chart there is a short “How to read this chart” note, and that note appears below every chart on the page. The idea was to make the dashboard usable without a separate guide, since not every user is expected to have a data analysis background.



Figure 9. Top Risk Segments chart highlighting the student groups with the highest dropout rates, displayed as colour-coded bars with their corresponding counts and rates.

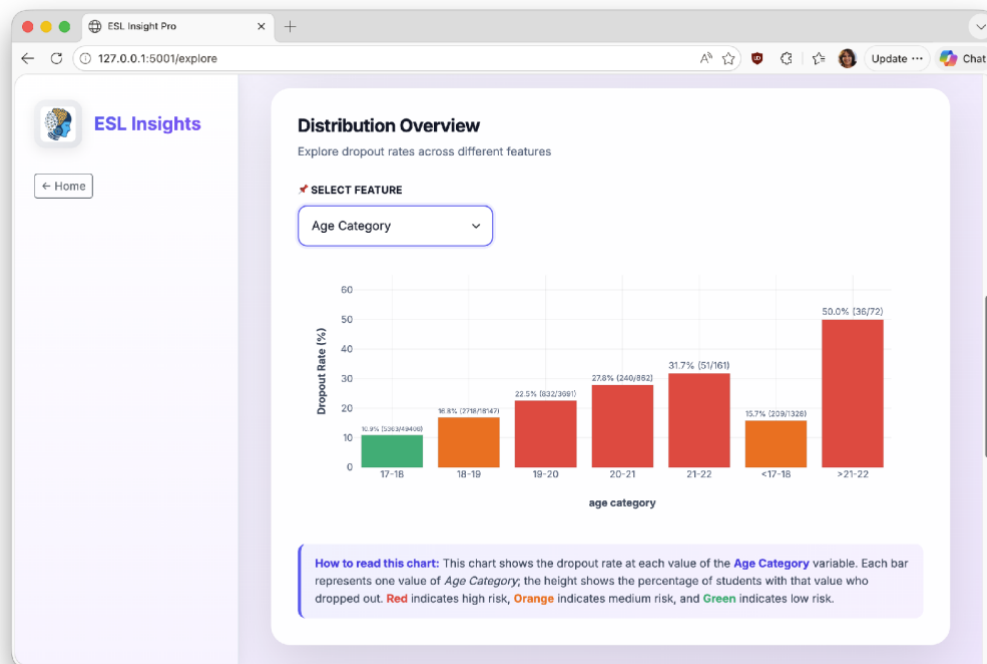


Figure 10. Distribution of dropout rates across age categories, showing lower-risk groups such as “17–18” (10.9%) in green, medium-risk such as “<17-18” (15.7%) in orange, and higher-risk groups such as “>21-22” (50%) in red.

The second part is the **Distribution Overview**, as shown in Figure 10 and Figure 11. Here the user picks a single feature from a dropdown, and the chart shows the dropout rate for each value of it. Every bar is one value of the feature, and it carries both the percentage and the raw counts, for example the number of dropouts over the number of students in that group. The bars are coloured the same way as the risk segments, so the high-risk values stand out at a glance. Putting the counts next to the percentage again

helps here, since it shows whether a high rate comes from a large group or a small one. This is the view for asking a focused question, like how dropout differs across the categories of one variable.

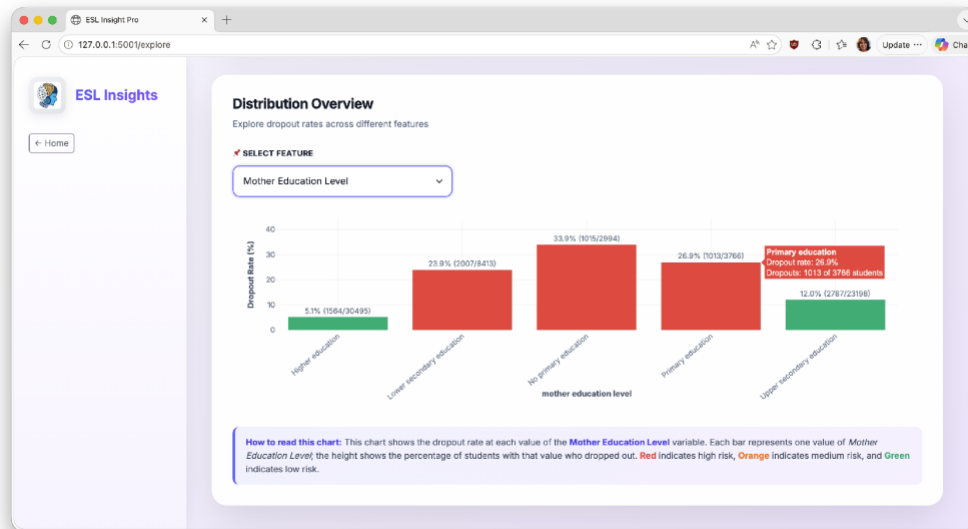


Figure 11. Distribution of dropout rates across mother education levels, showing lower-risk groups such as “Higher education” (5.1%) and “Upper secondary education” (12%) in green, and higher-risk groups such as “No primary education” (33.9%), “Primary education” (26.9%) and “Lower secondary education” (23.9%) in red.

The third part is **Custom Analysis**, which is more open-ended, as shown in Figure 12 and Figure 13. The user makes three choices from dropdowns: a feature for the x-axis, a metric for the y-axis, and a chart type, then presses Generate. The metric is one of three: total population, dropout, or enrolled. The chart type can be a bar, a pie or donut, or an area chart. So, the same feature can be looked at in different ways, as shown in Figure 14 and Figure 15. The user could also look, for instance, as the share of the total population or as the dropout count, depending on the question. This part was meant to give some room to explore beyond the fixed charts above it, without needing to touch the data directly.

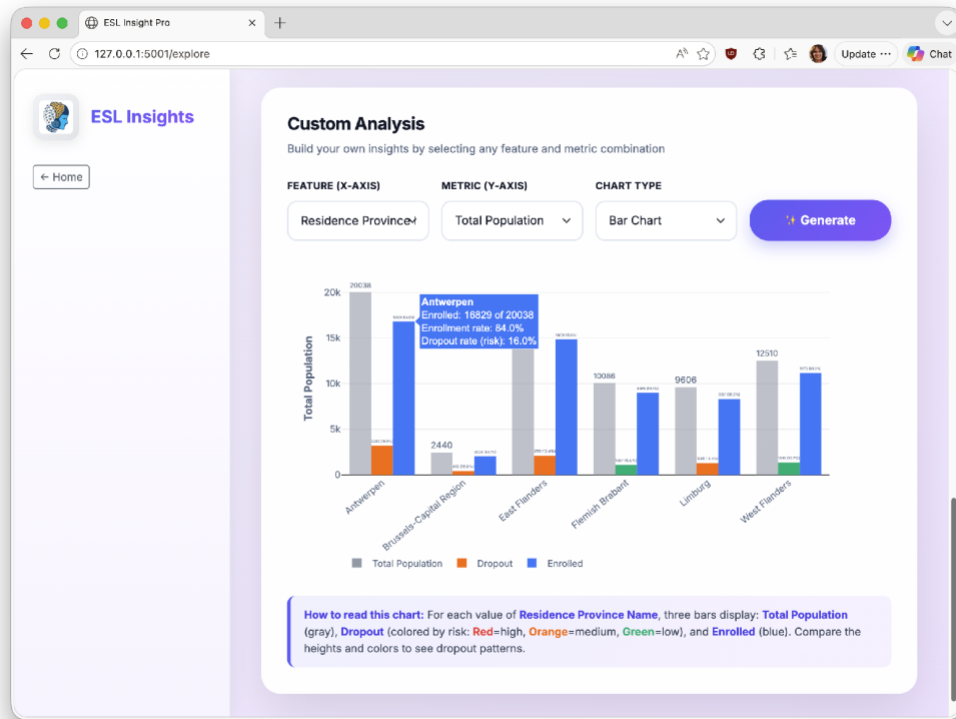


Figure 12. Custom analysis: the user picked the x-axis feature “Residence Province”, the y-axis metric “Total Population”, and the chart type “Bar Chart”.

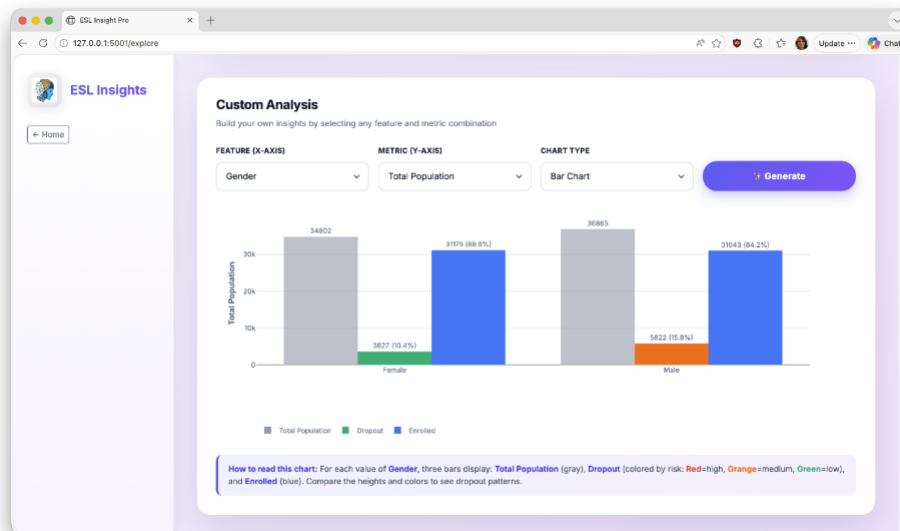


Figure 13. Custom analysis: the user picked the x-axis feature “Gender”, the y-axis metric “Total Population”, and the chart type “Bar Chart”.

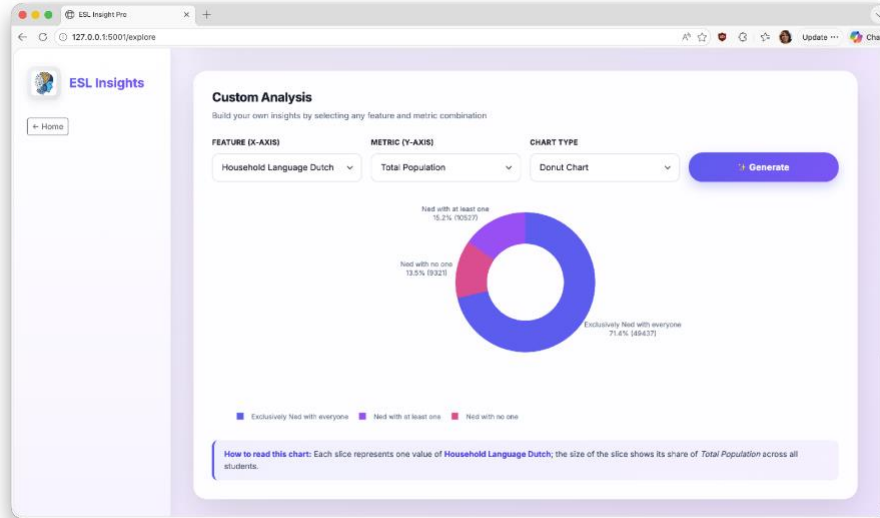


Figure 14. Custom analysis: the user picked the x-axis feature “Household Language Dutch”, the y-axis metric “Total Population”, and the chart type “Donut Chart”.

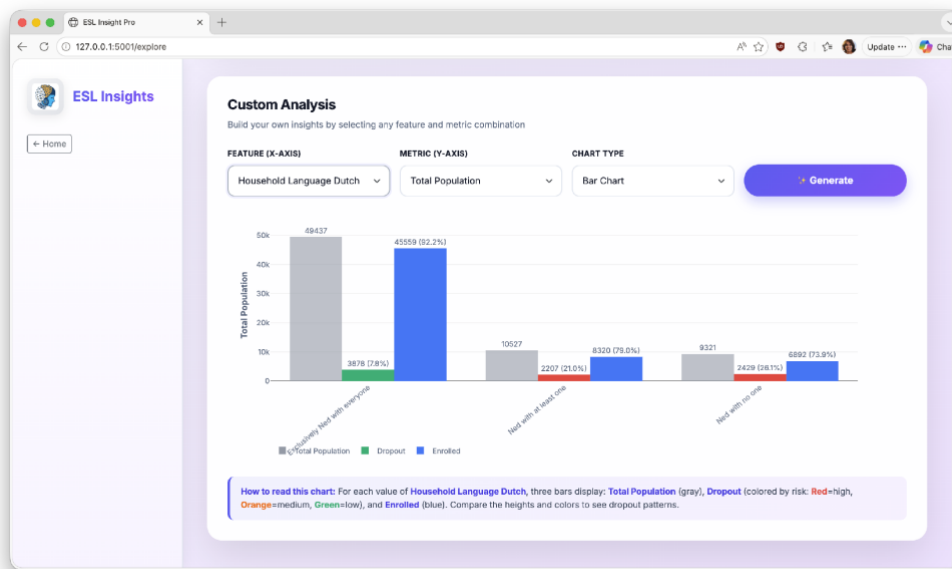


Figure 15. Custom analysis: the user picked the x-axis feature “Household Language Dutch”, the y-axis metric “Total Population”, and the chart type “Bar Chart”.

Taken together, the three parts move from a fixed summary to a focused look at one feature, to a free form view the user builds. The riskier groups are easy to spot because the dashboard highlights the highest dropout rates and colours them by severity, which turns a big table of numbers into a shorter list of groups worth a closer look.

6.3 Training models

Training has its own page, kept separate from prediction. It is laid out as a short top-to-bottom sequence, so the user moves through it in the same order each time: upload a dataset, set up the training, start it, and then manage the trained models.

On the training page the user picks a country, a target column, and a model, as shown in Figure 16. The system trains it, evaluates it, and stores it with its metadata: the country, the dataset name, the target column, the feature list, the model type, and the evaluation metrics.

Users can also upload another dataset that the model will be trained on. Once the file is read, its columns are pulled out and used to fill the target selection, so the user does not have to type any column names by hand. The second step is the training configuration. Here the user makes three choices: the country the dataset belongs to, the target column that marks dropout against still enrolled, and the model to train. Asking for the country here is what keeps the country-specific approach intact, since the trained model is stored against the country whose data it learned from.

The system allows users to train five different classifiers and activate the one that performs best for their data. The available models are logistic regression, decision tree, random forest, neural network, and XGBoost, and a concise description of each is provided in the subsections that follows. Each model offers distinct strengths, and providing multiple options makes it possible to compare different approaches on the same dataset. This flexibility is essential because the country datasets differ in size, structure, feature availability, and data quality. A model that performs well for one country may not behave the same way for another, so the system supports selecting and activating the most suitable model for each dataset.

Logistic Regression

Logistic regression is a supervised machine learning model commonly used for classification tasks. It is especially suitable for binary classification, where the outcome has two possible classes. In this study the two possible classes are dropout and non-dropout. The model estimates the probability that a student belongs to the dropout class. It does this by learning the relationship between the input features and the target class. Logistic regression is often used as a baseline model because it is simple, fast and easier to interpret than many complex models. One advantage of logistic regression is that it gives a clear starting point for comparison. If a more complex model does not perform better than logistic regression, then the added complexity may not be useful. Another advantage is that its results are easier to explain, because the model is based on a relatively simple decision structure. The main limitation is that logistic regression assumes a mostly linear relationship between the input features and the target outcome. Early school leaving is usually influenced by many factors at the same time. For this reason, regression may not always capture the full pattern in the data.

Model implemented: LogisticRegression from the Scikit-learn library¹

Decision Tree

A decision tree is a supervised machine learning model that makes predictions by splitting the data into branches. Each split is based on a feature value. The model continues splitting the data until it reaches a

¹ https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

final decision. Decision trees are useful because their structure is easy to understand. A prediction can be followed through the tree from the split to the final leaf. This makes the model more transparent than many other machine learning models. It is useful in the context of this system because it is not only expected to predict the risk but also help users understand why a student may be at risk. The main weakness of the decision tree is that they can be sensitive to noise in the data. A tree may learn patterns that are specific to the training data but do not generalize well to new data. This is known as overfitting, which matters because the datasets in this study differ in size and quality.

Model implemented: DecisionTreeClassifier from the Scikit-learn library²

Random Forest

Random forest is an ensemble model based on decision trees. Instead of using one tree, it builds many trees and combines their predictions. This usually gives more stable results than a single decision tree. The main advantage of random forest is that it can capture more complex patterns while reducing the risk of overfitting. Since each tree is trained with some randomness, the model does not rely too much on one specific structure. The final prediction is based on the combined output of many trees. Random forest is useful for tabular datasets like the ones used in this study. It can handle different types of variables and can work well when there are many features. This made it a suitable model for computing the dropout prediction performance across the country dataset. The main limitation is that the random forest model is less transparent than a single decision tree model. It is harder to explain the full decision process because the final prediction comes from many trees. For this reason, explainability methods were needed to make the prediction easier to understand.

Model implemented: RandomForestClassifier from the Scikit-learn library³

Neural Network

A neural network is a supervised learning model that can learn complex and non-linear relationships in the data. It is made up of layers. The input layer receives the features, one or more hidden layers learn patterns, and the output layer produces the prediction. Neural networks can be useful when the relationship between the features and the outcome is complex. This is relevant for early school leaving because dropout risk is not usually caused by one simple factor. It can be shaped by a combination of academic, social, family and contextual factors. The advantage of a neural network is its flexibility. It can learn patterns that simpler models may miss. This made it useful to include as part of the model comparison. The main limitation is interpretability. Neural networks are often harder to explain than models such as logistic regression or decision tree. The internal decision process is not easy to follow.

Model implemented: MLPClassifier from the Scikit-learn library⁴

XGBoost

XGBoost is a gradient boosting model based on decision trees. It builds trees step by step. Each new tree tries to correct errors made by the previous trees. This makes the model strong for many structured prediction tasks. XGBoost is often effective on tabular data. It can capture non-linear patterns and

² <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

³ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

⁴ https://scikit-learn.org/stable/modules/neural_networks_supervised.html

interactions between variables. This was useful for the dataset because the relationship between student background, school pathway and dropout risk may not be simple. Another advantage of XGBoost is that it includes regularization techniques. These help reduce overfitting and make the model more stable. This is important when working with dataset that differ in size and quality. The main limitation is that XGBoost is more complex than simple models. Its predictions are not easy to explain directly. Because of this, explainability methods were needed to show which features contributed most to each prediction.

Model implemented: XGBClassifier from the xgboost library⁵

Next to these choices there is an **Advanced Settings** option. It opens a small dialog with the main hyperparameter for the selected model, and the set of fields changes with the model: for XGBoost it shows things like the number of boosting rounds, the learning rate, and the tree depth, while for the neural network it shows the hidden layer sizes and the activation function. Each field comes with a sensible default, so a user can train a model straight away without touching them, but the option is there for anyone who wants to tune the model. Pressing **Start Training** then trains the model, evaluates it on the held-out test data, and stores it together with its metadata: the country, the dataset name, the target column, the feature list, the model type, and the evaluation metrics.

Figure 17 shows the **Advance Settings** dialog for XGBoost, with its boosting rounds, learning rate, tree depth, subsample, and column-sampling fields filled in with their defaults. Since most expected users are not machine-learning experts, we recommend keeping the default values. Advanced users with Machine Learning knowledge can adjust these parameters as needed.

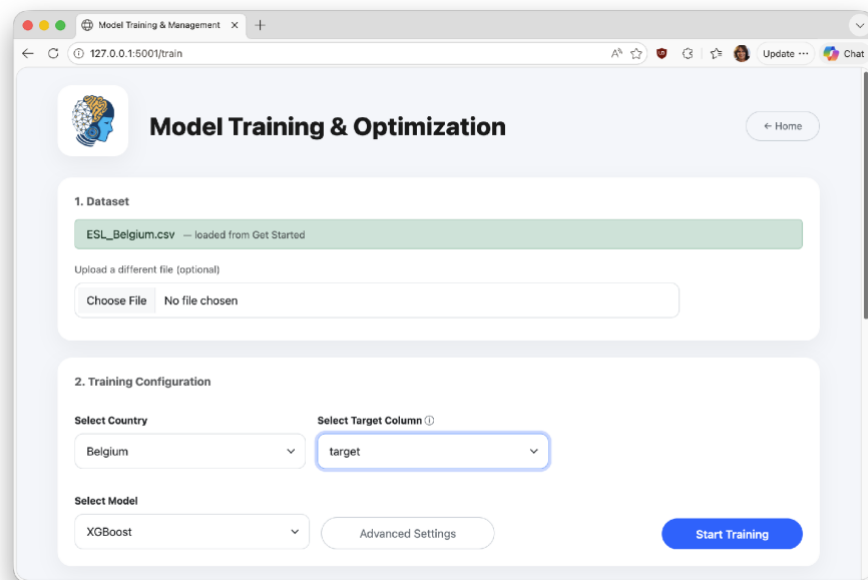


Figure 16. The training page is laid out as a fixed top-to-bottom sequence (upload dataset, configure training, start, manage models) so the steps cannot be taken out of order. The part to focus on is the configuration block, where the user picks the

⁵ https://xgboost.readthedocs.io/en/stable/get_started.html

country, the target column, and one of the five models: choosing the country here is what enforces the country-specific approach, since each trained model is stored and later activated against the dataset it learned from.

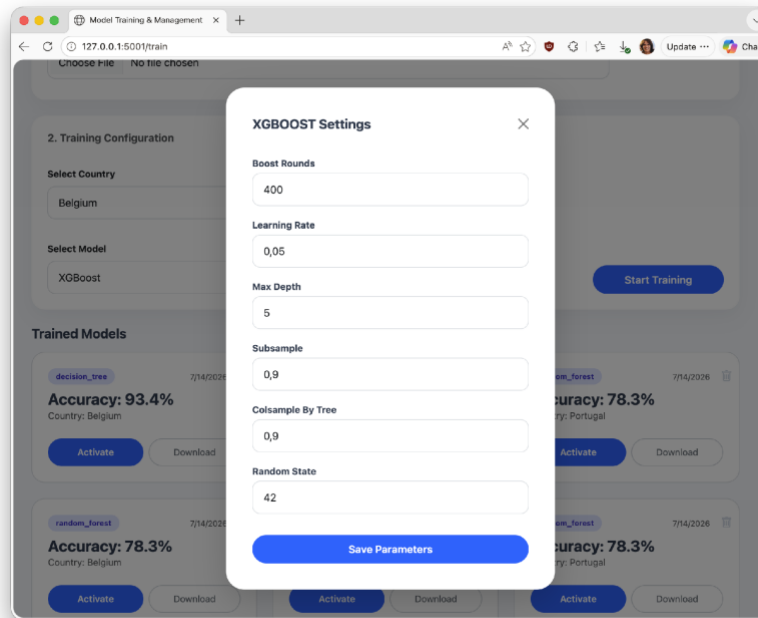


Figure 17. Advanced settings dialog for XGBoost. The point to notice is that every hyper-parameter comes pre-filled with a sensible default, so a non-expert can train a model without touching it. Advance settings should only be used by users with machine-learning knowledge. If you are not familiar with the meaning of the model parameters, avoid changing them and keep the default values.

The lower part of the page lists the trained models, as shown in Figure 18. Each one shows up as a card with its model type, its accuracy, the country it was trained for, and the date it was created. From a card the user can activate the model, download it, or delete it. Activation is the key action here: only one model is active at a time, the active one is highlighted, and that is the model used on the prediction page. Each country keeps its own model and a prediction only runs against the model whose feature structure fits the input. When deciding which model to activate, always look at the accuracy and other evaluation metrics: models with higher accuracy are more likely to produce better predictions. This follows the country-specific approach, where each dataset keeps its own structure instead of being forced into a single universal model.

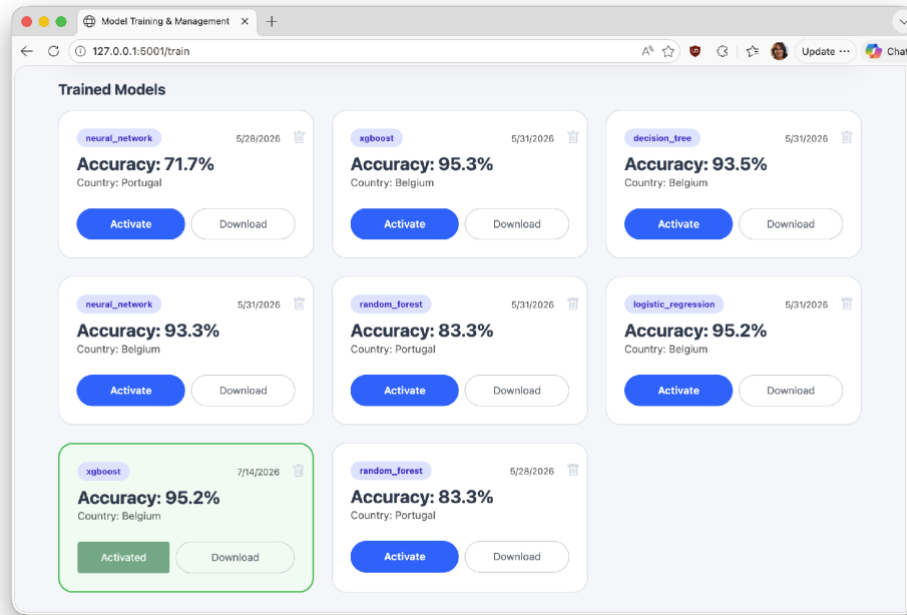


Figure 18. Only one model can be active at a time, and the active model is the one used for prediction. You must activate the model that was trained with the data from the country for which you want to generate predictions.

6.4 Generating predictions

The last step is prediction, but keep in mind that an activated model is required, as shown in Figure 19. The user gives one or more student records, and for each student the system returns a predicted outcome, a dropout probability, the main factors behind the prediction, and a readable summary with a suggested direction for support. The results show up in a table and can be downloaded. So, the workflow runs from raw data all the way to an explainable prediction, with each step still understandable on its own.

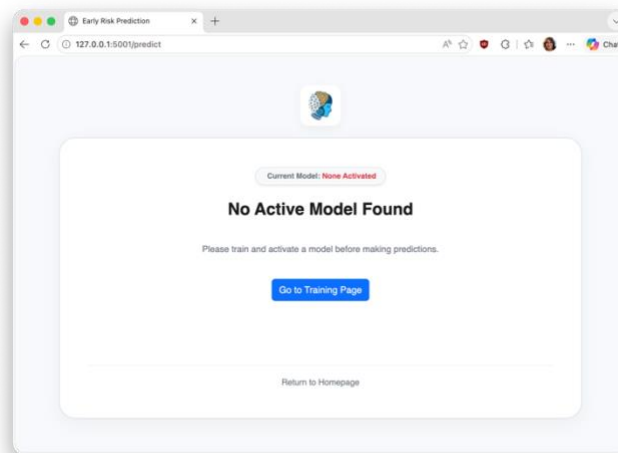


Figure 19. An activated model is required to be able to make predictions, otherwise you will get this message.

Prediction can be done in two ways: upload a CSV with several student records for batch prediction or enter each single student by hand (only available for Belgium). Before either of those, the prediction page asks a couple of setup questions, so the manual path is really a short wizard rather than one form. Figure 20 shows the first screen asking which country the school is in, and it shows the model that is currently active at the top. This acts as a reminder that the active model decides which features will be asked for. It also helps the user check whether the selected country and active model are consistent.

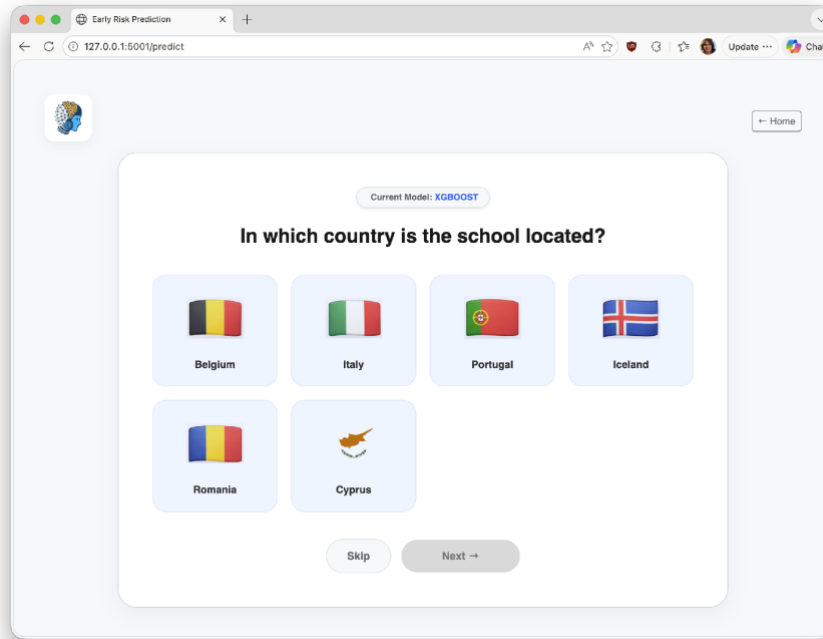


Figure 20. Choosing the country, with the active model shown at the top. In this example, the active model XGBoost.

The screen shown in Figure 21 asks how the data should be entered. Here the user chooses between manual entry and CSV upload, and there is also a toggle for the AI explanations. With the toggle on, the readable summaries are generated by the local language model through Ollama⁶; with it off, the system still returns the prediction and the factors but skips the generated text. Having this switch matters in practice, since the summaries only work when Ollama is available locally and turning them off keeps the rest of the tool usable when it is not.

⁶ <https://ollama.com/>

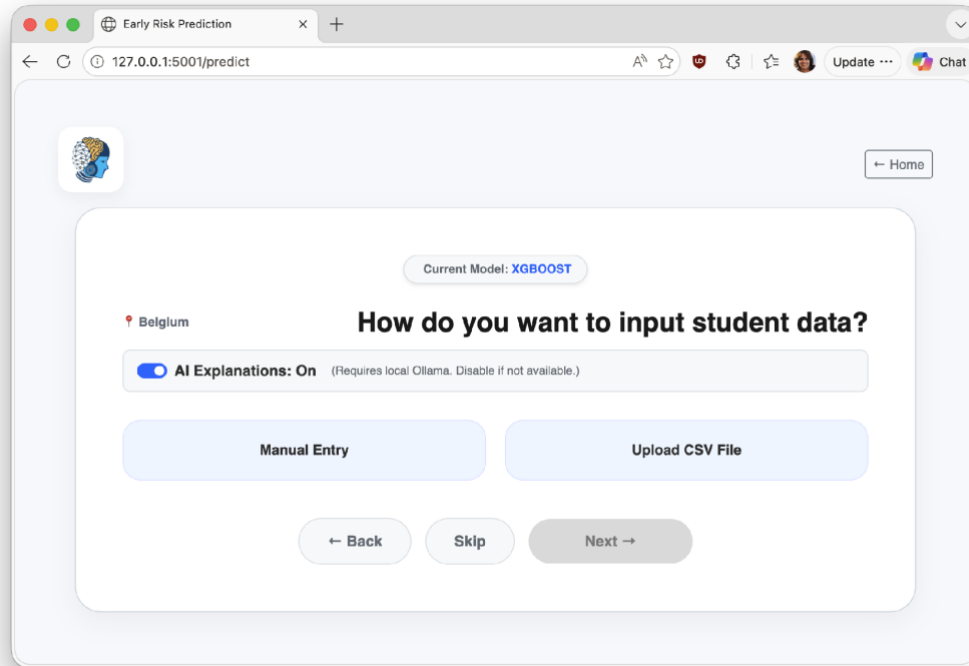


Figure 21. Choosing how to enter the data, with the toggle for local AI explanations.

Manual entry

If manual entry is chosen, the system first asks how many students will be entered, as shown in After that it walks through the features one at a time. Each feature is its own question, usually with a set of options to pick from, and a small badge shows which student is being filled in, as shown in Figure 22. The answers are collected step by step until there is a full profile for each student, and only then does it go to the model.

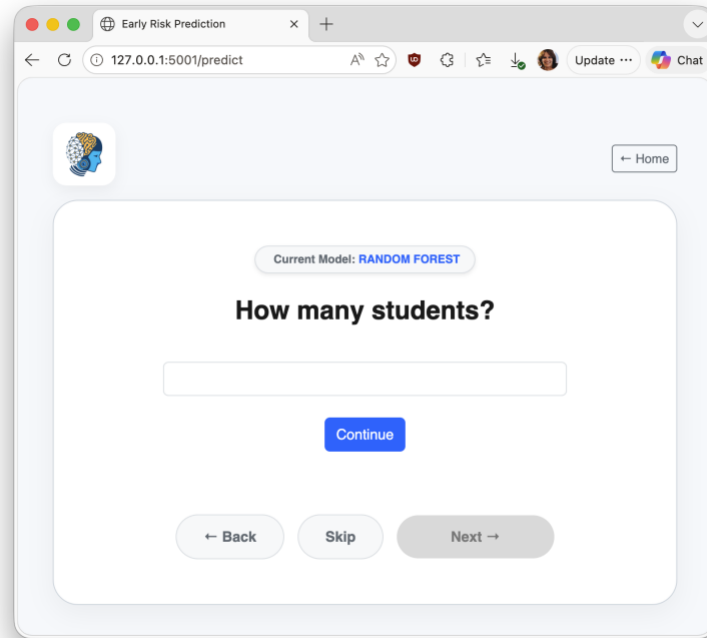


Figure 22. Asking how many students will be manually entered.

Adding this manual option was not only about convenience. Filling in each feature value by hand opens up a few things that turned out useful for understanding, testing, and showing the system. One is controlled experimentation. Since the user sets each value directly, it is easy to try out different student profiles and watch how the prediction reacts.

You can ask practical questions, like what happens to the risk if one factor changes and the rest remain unchanged. That shows how the model behaves, not just what it says for a single fixed record. For a decision-support tool that kind of testing is helpful, because it takes the model's behavior visible instead of hiding it behind one number.

The second is transparency. Going through the features one by one makes the logic easier to grasp. The user sees exactly which variables the model uses and can tie them back to the explanation that comes with the prediction. That fits the explainability direction of the thesis, since it links what the user typed in with the factors the system reports as important.

The third is communication. Manual mode is a simple way to show the system to people who are not technical. In a demo you can enter different profiles live and let the audience watch the predicted risk shift from one to the next. That makes the tool easier to explain to teachers, school staff, or policymakers, and it supports the point raised in the stakeholder discussion, that the system should help people reflect on risk rather than just give them only a label. These manual experiments need to be read with care. Changing a value and seeing the prediction move tells you how the model responds to the data, not that the factor causes dropout in real life.

So manual mode is most useful for understanding and explaining the system, and the results still have to be read together with human judgement and what people actually know about the situation. The following figures (from Figure 23 to Figure 28) show some screens of how to manually enter a single student to be predicted.

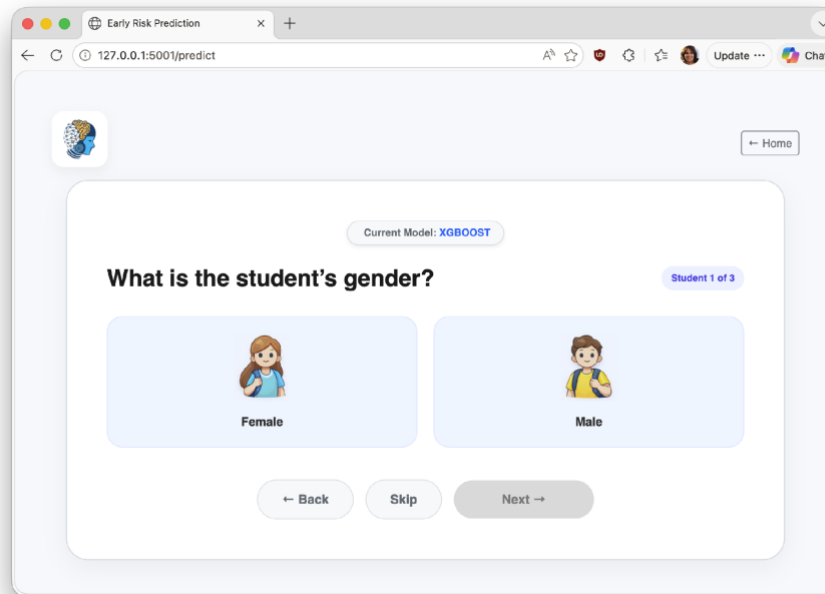


Figure 23. One feature as its own question, with a badge for the current student. In this example, the student's gender.

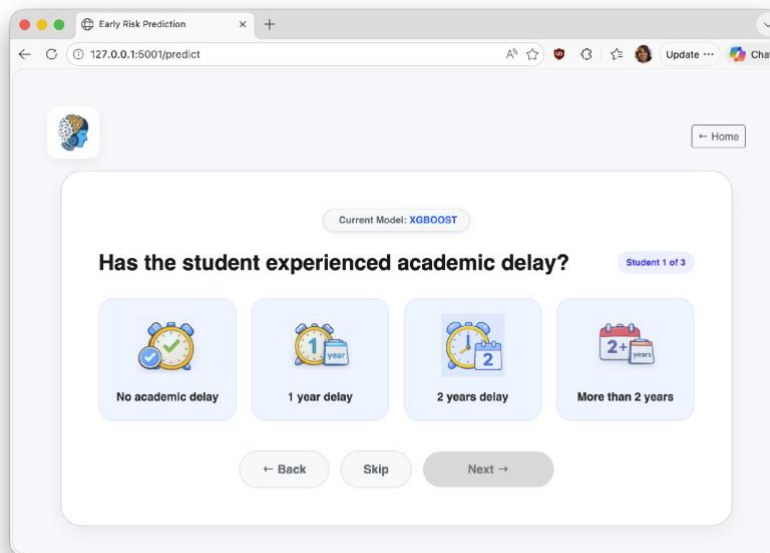


Figure 24. One feature as its own question, with a badge for the current student. In this example, the experienced academic delay.

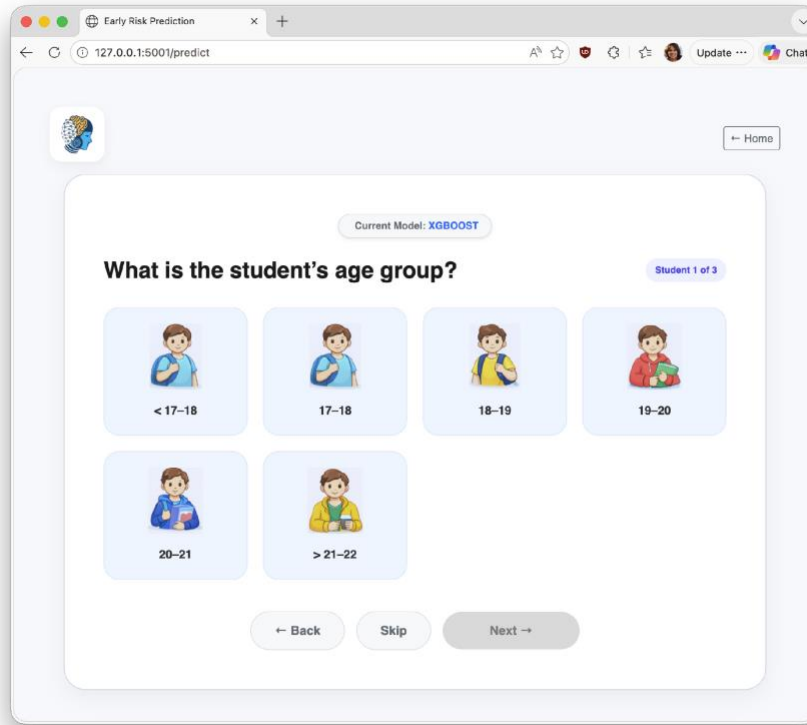


Figure 25. One feature as its own question, with a badge for the current student. In this example, the student's age group.

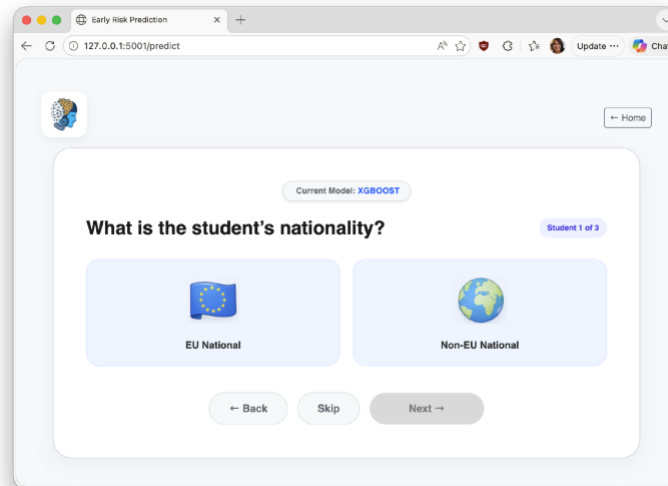


Figure 26. One feature as its own question, with a badge for the current student. In this example, the student's nationality.



Figure 27. One feature as its own question, with a badge for the current student. In this example, the mother's educational level.

Figure 28. One feature as its own question, with a badge for the current student. In this example, the screen asks: Does the student live in a central city?

Upload a CSV file

If the option **Upload a CSV File** is chosen, the system allows the possibility to upload a CSV file with all students the user wants to predict, and then go to prediction, as shown in Figure 29.

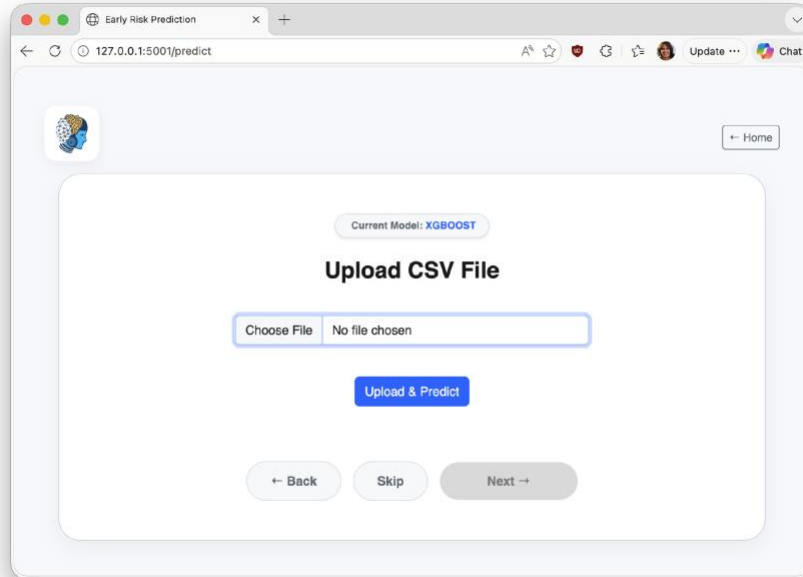


Figure 29. One feature as its own question, with a badge for the current student. In this example, the screen asks: Does the student live in a central city?

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	birth_country	gender	nationality	distance_kilometrage	age	state_support	responsible_person	education_responsible_person	father_employment_status	father_education_level	mother_employment_status	mother_education_level	educational_track
2	Cape Verde	F	Cape Verde	Short	17	Yes	Other	Master	Employee	Master	Employee	Low Secondary	General
3	Cape Verde	M	Cape Verde	Short	18	Yes	Mother	Unknown	Employee	Unknown	Domestic (no job)	Unknown	Technical
4	Portugal	M	Portugal	Moderate	17	Yes	Other	Low Secondary	Unemployed	Low Secondary	Employee	Low Secondary	Technical
5	Portugal	M	Portugal	Short	17	Yes	Mother	Low Secondary	Employee	Primary	Unemployed	Low Secondary	General
6	Portugal	F	Portugal	Moderate	17	Yes	Mother	Master	Employee	Upper Secondary	Employee	Master	General
7	Portugal	F	Portugal	Long	18	Yes	Mother	Unknown	Employee	Low Secondary	Domestic (no job)	Unknown	General
8	Portugal	F	Portugal	Short	17	Yes	Mother	Upper Secondary	Employee	Upper Secondary	Business owner	Upper Secondary	General
9	Portugal	M	Portugal	Moderate	18	Yes	Mother	Master	Employee	Master	Employee	Master	General
10	Portugal	F	Portugal	Moderate	18	Yes	Mother	Upper Secondary	Employee	Low Secondary	Employee	Upper Secondary	General
11	Portugal	M	Portugal	Short	18	Yes	Mother	Master	Employee	Master	Unemployed	Master	General
12	Portugal	M	Portugal	Long	17	Yes	Mother	Master	Employee	Low Secondary	Unknown	Master	General
13	Portugal	F	Portugal	Short	17	Yes	Mother	Upper Secondary	Employee	Upper Secondary	Employee	Upper Secondary	General
14	Brazil	F	Portugal	Short	17	No	Mother	Upper Secondary	Unknown	Unknown	Employee	Upper Secondary	General
15	Portugal	M	Portugal	Moderate	17	Yes	Mother	Upper Secondary	Unknown	Upper Secondary	Business owner	Upper Secondary	General
16	Portugal	M	Portugal	Long	17	Yes	Mother	Upper Secondary	Business owner	Upper Secondary	Domestic (no job)	Upper Secondary	Technical
17	Portugal	M	Portugal	Moderate	17	Yes	Mother	Upper Secondary	Unknown	Unknown	Employee	Upper Secondary	General
18	Portugal	F	Portugal	Long	17	No	Other	Low Secondary	Employee	Low Secondary	Employee	Upper Secondary	General
19	Portugal	F	Portugal	Short	17	No	Mother	Upper Secondary	Employee	Low Secondary	Employee	Upper Secondary	General
20	Portugal	M	Portugal	Short	17	Yes	Mother	Upper Secondary	Employee	Low Secondary	Employee	Upper Secondary	General
21	Portugal	M	Portugal	Long	17	Yes	Mother	Upper Secondary	Employee	Low Secondary	Employee	Upper Secondary	General

Figure 30. Example of a list of students to be predicted. Notice that each row corresponds to a single student and each column represents a feature (variable or characteristic) describing that student.

The dataset used for prediction must be provided as a CSV file, where each row represents a single student and each column corresponds to a feature describing that student, as shown in Figure 29. It is essential to use the same list of features that were included in the training dataset. Any missing, additional, or renamed features will prevent the system from generating valid predictions.

Interpreting predictions

Prediction results are shown with the explanation and suggested support direction, as shown in Figure 29.

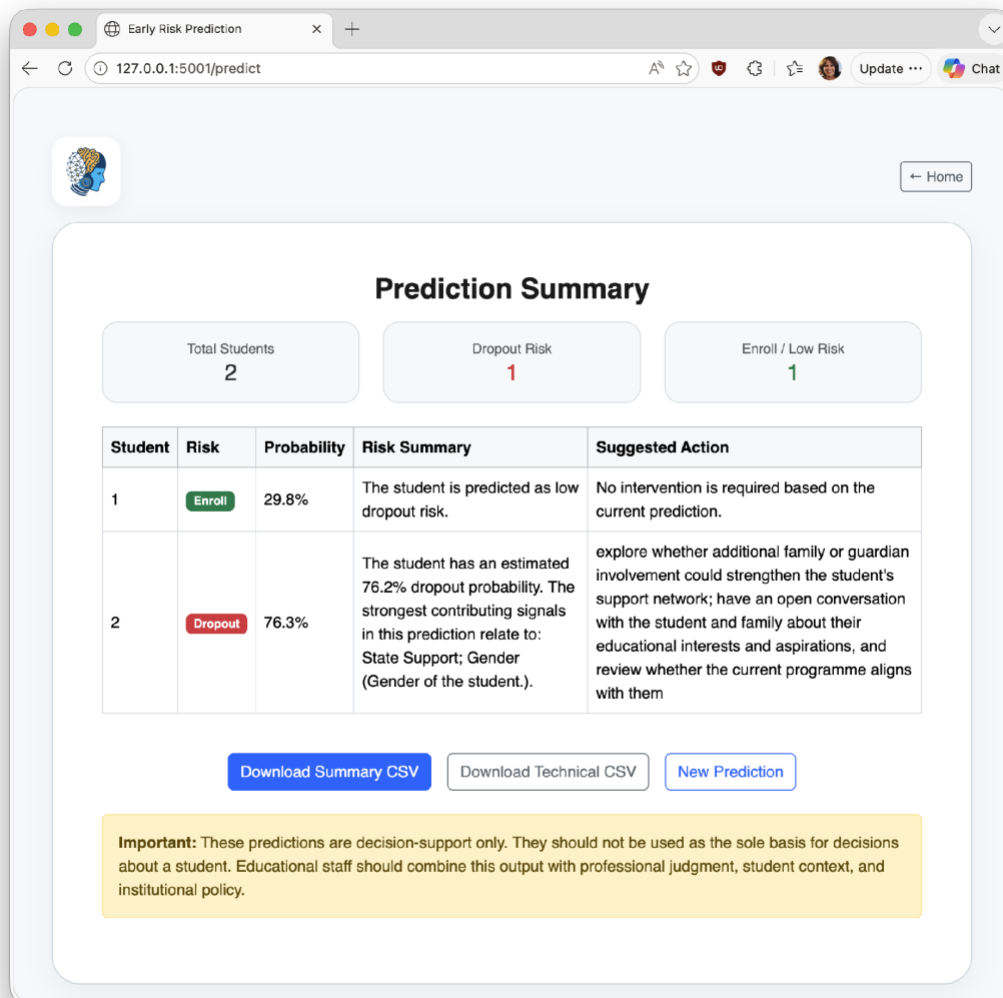


Figure 29. Prediction results with the explanation and suggested support direction. Each prediction is presented together with its dropout probability, the main contributing factors, and a readable support suggestion, which is what turns the result into something an educator can question and act on.

The main role of explainability in this system is to make the model output easier to interpret. A prediction such as “dropout risk” is too limited on its own. It does not show why the model reached that result. For this reason, the system combines the prediction with an explanation. Each prediction includes the predicted class, the probability score, and the main factors that influenced the result. This makes the

output more useful for practical decision-making. The explanations are not intended to be final conclusions about students. They are support signals. They can help educators notice possible risk areas, but the final interpretation should always include human judgment and knowledge of the student's real situation. As the data does not give the full picture of early school leaving, this is really important. Some of the important factors may be missing from the data. Motivation, mental health, family problems, school climate, and teacher-student relationships may all matter, but they are not always available in the datasets.

The Feature importance helps show which variables have the strongest influence on model predictions. At the individual level, feature importance shows which factors mattered most for one student prediction. This is useful because two students can both be predicted as at risk, but for different reasons. One student's prediction may be influenced more by academic progress, while another student's prediction may be influenced more by family or background variables. At a broader level, feature importance can also help identify patterns across many predictions. If the same variables often appear as strong risk factors, this may indicate wider patterns in the dataset. For example, school progress, absences, or certain background indicators may repeatedly appear as important signals. However, feature importance must be interpreted carefully. A feature being important in the model does not mean that it is the direct cause of dropout. It only means that the model found a useful pattern in the available data. This distinction is important because the system is based on historical data, not on a full causal explanation of each student's situation.

Individual risk explanations are central to the prototype. The system is designed to support student level interpretation, not only general model analysis. For each student, the system returns a prediction and a probability score. The prediction shows whether the student is classified as dropout risk or not. The probability gives an estimate of how confident the model is in that prediction. The system then adds the main explanation factors. In the implemented workflow, the top contributing factors are extracted for each prediction. Each factor is shown with a direction, such as increasing or decreasing dropout risk. This makes the result easier to understand than a probability score alone. The feature names are also cleaned before being presented because raw model outputs can contain technical names from preprocessing, such as encoded categorical variables. These names are not always readable for educators. The system therefore simplifies feature names where possible. An individual explanation could therefore take the following form: the student is predicted as at risk, the estimated dropout probability is high, and the main contributing factors are linked to school progress, language background, or another available risk indicator. This gives the user a starting point for further reflection. These explanations should not be used as automatic decisions. They should be used as support for early attention. A high-risk prediction can help educators decide which cases may need closer review, but it should always be checked against the real situation of the student. In this way, explainability supports the main goal of the thesis. The system does not only identify students at risk. It also gives users a clearer view of the factors behind the prediction, making the output more transparent and more useful for educational decision support.